



Gymnasium Trudering

Naturwissenschaftlich-technologisches und Sprachliches Gymnasium

Seminararbeit

zum Thema

Analyse von neuronalen Netzen am Beispiel der Bilderkennung

Rahmenthema: **Data Science**

Leitfach: **Mathematik**

Verfasser: **Daniel Shamsi**

Kursleiter: **Josef Lächele**

Abgabetermin: **8.11.2022**

Art	Note	Note in Worten	Punkte	Gewichtung	Gesamtpunktzahl
Seminararbeit				3	
Abschlusspräsentation				1	

Summe	
--------------	--

Gesamtleistung nach § 29 Abs. 7 GSO Summe dividiert durch zwei und anschließend gerundet	
--	--

.....
Datum und Unterschrift des Kursleiters

Zusammenfassung

Während klassische regelbasierte Programmierung bei Bilderkennung scheitert, bieten neuronale Netzwerke die Möglichkeit, große Datenmengen zu nutzen, um das Problem mit hoher Genauigkeit zu lösen. (Niall O' Mahony, 2019; Andrew Brock, 2021). Die vorliegende wissenschaftliche Arbeit erklärt neuronale Netze am Beispiel der Bilderkennung. Insbesondere werden Faltungsnetzwerke beschrieben, die entscheidend für die Anwendung neuroyaler Netze auf Bilder sind. Im Rahmen dieser Arbeit wurde ein neuronales Netz programmiert, welches in der Lage ist Katzenbilder von Hundebildern mit einer Genauigkeit von 87,3% zu unterscheiden.

Inhaltsverzeichnis

1 Einleitung	1
2 Künstliche neuronale Netze	2
2.1 Perzeptron.....	2
2.2 Multilayer Perzeptron	2
2.3 Feedforward-Netze und Aktivierungsfunktionen.....	3
2.3.1 Sigmoid Funktion.....	4
2.3.2 ReLU Funktion	4
2.4 Maximum Likelihood Estimation.....	5
2.5 Optimizer	7
2.5.1 Gradientenabstiegsverfahren.....	7
2.5.2 Stochastisches Gradientenabstiegsverfahren	8
2.5.3 Momentum.....	9
2.5.4 RMSProp.....	9
2.5.5 Adam.....	10
2.6 Overfitting	10
3 Convolutional Neuronal Network	11
3.1 Eingabe	11
3.2 Faltungsschicht	12
3.2.2 Padding	13
3.2.3 Stride	14
3.3 Pooling.....	14
4 Praktisches Beispiel	15
4.1 Aufgabe	15

4.2 Daten.....	15
4.3 Verwendete Programmiersprache & Bibliotheken.....	16
4.4 Training	16
4.5 Ergebnis	17
5 Resümee.....	17
Literaturverzeichnis.....	18
Appendix A	20
A.1 Mathematischer Hintergrund	20
A.1.1 Matrizen.....	20
A.1.2 Gradient	21
Appendix B.....	21
B.1 Faltung	21
B.1.2 Kreuzkorrelation	22
Anhang	23
Erklärung.....	24

1 Einleitung

Was ist in Abb. 1 zu sehen? Während Menschen im Allgemeinen kein Problem haben, die Katze in dem Bild zu erkennen, ist es für Computer eine komplizierte Aufgabe. Das Problem hierbei ist es, nur anhand der Farbpixel im Bild eine Regel aufzustellen, die verlässlich zwischen einer Katze und einem Hund unterscheiden kann. Solche Aufgaben zeichnen sich durch ein hohes Maß an Variabilität aus, weshalb die klassische regelbasierte Programmierung daran scheitert. (Niall O' Mahony, 2019). Auf der anderen Seite gibt es im Internet große Datenmengen, die den Unterschied zwischen Katzen und Hunden zeigen. (Dogs vs. Cats Redux: Kernels Edition, 2016). Maschinelles Lernen versucht diese Datenmengen zu nutzen, um die Regeln implizit zu lernen. Insbesondere neuronale Netze werden im Laufe des letzten Jahrzehnts vermehrt auf das Problem der Bildklassifikation angewandt und erkennen mittlerweile einige Objekte in einzelnen Datensätzen besser als Menschen. (Kaiming He, 2015)



Abb. 1: Bild einer für den Menschen leicht zu erkennende Hauskatze. Aus dem Trainingsdatensatz (Dogs vs. Cats Redux: Kernels Edition, 2016).

In dieser Arbeit wird der Hintergrund von neuronalen Netzen vorgestellt und auf die Klassifizierung von Hunden und Katzen anhand eines Open Source Datensatzes angewandt. (Dogs vs. Cats Redux: Kernels Edition, 2016). Insgesamt erreicht das Modell eine Genauigkeit von 87,3% auf einem unabhängigen Test-Datensatz.

2 Künstliche neuronale Netze

Zuerst wird auf die historische Entwicklung und Struktur von neuronalen Netzen eingegangen. Es wird das Grundprinzip hinter dem Lernen und heutigen Optimierungs-Methoden erläutert. Abschließend wird aufgezeigt was passiert, wenn sich ein Netzwerk zu stark an die Trainingsdaten anpasst, und wie man das verhindern kann. Die Mathematischen Hintergründe zu neuronalen Netzen werden in Appendix A beschrieben.

2.1 Perzeptron

Um einen ersten Einblick in die Funktionsweise Künstlicher Neuronaler Netze zu erhalten, wird im Folgenden das einfachste neuronale Netzwerk beschrieben: Das Perzeptron, welches 1957 von Frank Rosenblatt eingeführt wurde, besteht aus einem einzigen Neuron mit einer binären Ausgabe und mehreren reellen Eingaben. (Rosenblatt F. , 1958, S. 386-408). Das Perzeptron berechnet dann die gewichtete Summe der Eingaben und klassifiziert die Eingabe als erste oder zweite Klasse abhängig davon, ob die Summer kleiner gleich oder Größer als Null ist. Für einen Eingabevektor $x \in \mathbb{R}^n$, den Gewichtsvektor $w \in \mathbb{R}^n$ und die Verschiebung $b \in \mathbb{R}$, kann das Perzeptron mathematisch als

$$f : \mathbb{R}^n \rightarrow \{0,1\}, \quad f(x) = \sigma(w^T x + b)$$

Formel 1: Perzeptron

definiert werden. σ ist hier die Aktivierungsfunktion, die positive Zahlen inklusive Null auf eins und negative Zahlen auf null abbildet.

$$\sigma : \mathbb{R} \rightarrow \{0,1\}, \quad \sigma(t) = \begin{cases} 0, & \text{wenn } t < 0 \\ 1, & \text{sonst} \end{cases}$$

Während des Lernens werden dann die Gewichte w verändert, um die Daten möglichst gut zu klassifizieren. Perzeptronen können für einfache lineare Klassifizierungsaufgaben genutzt werden. Sobald komplexere Funktionen benötigt werden, reicht ein einstufiges neuronales Perzeptron nicht mehr aus. (Bishop, 2009, S. 192-194)

2.2 Multilayer Perzeptron

Um komplexere Funktionen darstellen zu können, können mehrere Perzeptronen miteinander verkettet werden. Dabei besteht jede Schicht aus mehreren Perzeptronen, deren Ausgabe in

einen Vektor geschrieben wird. Dieser Vektor wird dann als Eingabe für die Perzeptronen der nächsten Schicht verwendet. Dabei kann die Anzahl der Perzeptronen in jeder Schicht frei gewählt werden. Um mehrere Perzeptronen in jeder Schicht darzustellen, kann Formel 1 mit Hilfe einer Matrixmultiplikation umgeschrieben werden:

$$f: \mathbb{R}^n \rightarrow \{0,1\}^m, \quad f(x) = \sigma(Wx + b)$$

wobei W eine $\mathbb{R}^{m \times n}$ -Matrix ist, $b \in \mathbb{R}^m$ ein Vektor ist und σ elementweise ausgeführt wird. Ein mehrschichtiges Perzeptron mit L Schichten und den Dimensionen $d_1, \dots, d_L$ in den Schichten $1, \dots, L$ kann mathematisch als

$$f: \mathbb{R}^n \rightarrow \{0,1\}^{d_L}, \quad f(x) = \sigma(W_L \sigma(W_{L-1} \sigma(\dots \sigma(W_1 x + b_1) \dots) + b_{L-1}) + b_L)$$

beschrieben werden. Dabei haben die Gewichtsmatrizen die Größen $W_1 \in \mathbb{R}^{d_1 \times n}$, $W_2 \in \mathbb{R}^{d_2 \times d_1}, \dots, W_L \in \mathbb{R}^{d_L \times d_{L-1}}$ und $b_1 \in \mathbb{R}^{d_1}$, $b_2 \in \mathbb{R}^{d_2}, \dots, b_L \in \mathbb{R}^{d_L}$. Die Anzahl der Schichten wird auch als Tiefe des neuronalen Netzes bezeichnet. Insgesamt ist das mehrschichtige Perzeptron also eine Verkettung von mehreren mehrdimensionalen linearen Funktionen mit der Aktivierungsfunktion σ . Je mehr Ebenen mit Neuronen vorhanden sind, desto komplexere Klassifizierungsaufgaben können erfüllt werden. Eine Schicht wird auch als fully-connected Layer bezeichnet, weil jede Eingabe Einfluss auf jede Ausgabe hat und damit die Eingaben vollständig mit den Ausgaben verbunden sind. Damit das neuronale Netz funktioniert, muss in jedem Neuron das richtige Gewicht gefunden werden. Um dies zu schaffen, wird das Gewicht so lange geändert, bis die geeignete Verlustfunktion möglichst klein wird. (Goodfellow, 2016, S. 170-172)

2.3 Feedforward-Netze und Aktivierungsfunktionen

Ein Feedforward Neuronales Netzwerk ist eine Verallgemeinerung des Multilayer Perzeptrons, bei dem andere Aktivierungsfunktionen verwendet werden. Eine Schicht in einem Feedforward Netz wird auch als fully-connected Layer bezeichnet. Diese Aktivierungsfunktionen sollten nichtlinear sein, da sonst das neuronale Netz eine Zusammensetzung linearer Funktionen und damit selbst linear ist. Außerdem ist es für das Lernen hilfreich, wenn die Funktion differenzierbar ist (siehe 2.5.1 Gradientenabstiegsverfahren).

2.3.1 Sigmoid Funktion

Die Aktivierungsfunktion von mehrschichtigen Perzeptronen ist am Nullpunkt nicht differenzierbar und ihre Ableitung ist für alle anderen Punkte gleich Null. Damit kann die Funktion aber relativ schlecht genutzt werden um ableitungsbasierte Optimierungsalgorithmen wie das Gradientenabstiegsverfahren (siehe 2.5.1 Gradientenabstiegsverfahren) anzuwenden. Als differenzierbare Approximation der Funktion wird deshalb oft die Sigmoid Funktion verwendet:

$$f: \mathbb{R} \rightarrow (0,1), \quad f(x) = \frac{1}{1 + e^{-x}}$$

Sie bildet die reellen Zahlen auf das Intervall $(0, 1)$ ab, wobei sie nahe bei null für kleine Werte und nahe bei eins für große Werte ist. Null wird auf 0.5 abgebildet. (Aston Zhang, 2021, S. 174-175)

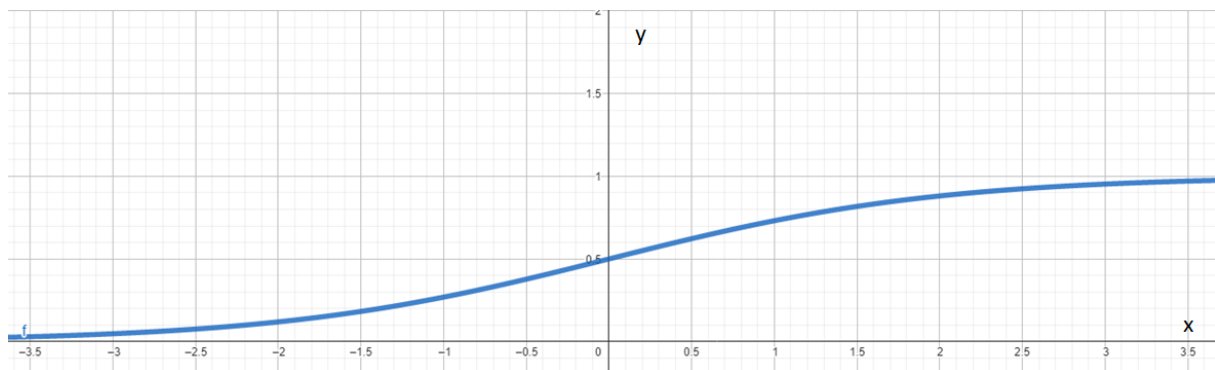


Abb. 2: Graph der Sigmoid Funktion

2.3.2 ReLU Funktion

Ein Problem mit der Sigmoid Aktivierungsfunktion ist, dass ihre Ableitung für kleine und große Werte nahe null ist. Durch Rundungsfehler am Computer, kann das besonders bei tiefen neuronalen Netzen zu Problemen führen. Eine Alternative ist die ReLU (Rectified Linear Unit) Funktion:

$$f: \mathbb{R} \rightarrow [0, \infty], \quad f(x) = \max(0, x) = \begin{cases} x, & \text{wenn } x \geq 0 \\ 0, & \text{sonst} \end{cases}$$

Die Aktivierung ist linear für Eingabe Werte größer als null. Negative Werte werden zu null und positive Werte werden übernommen. (Aston Zhang, 2021, S. 173-174)

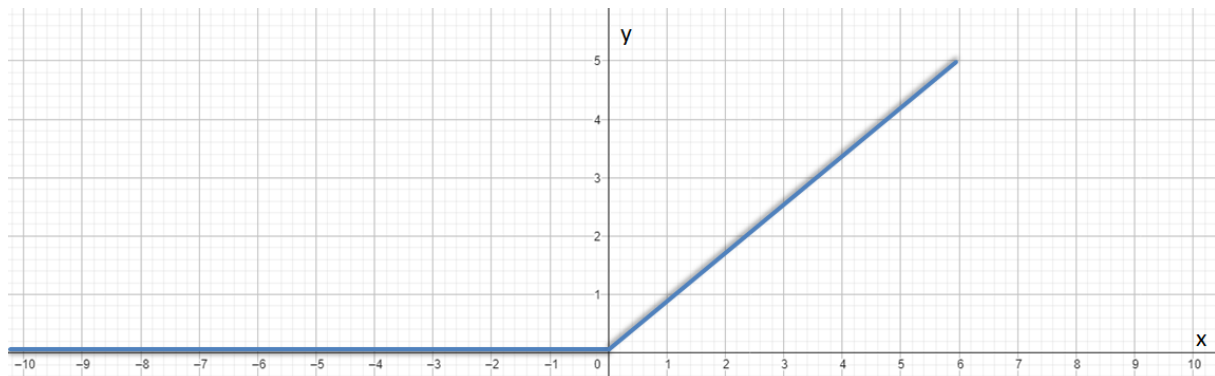


Abb. 3: Graph der ReLu Funktion

2.4 Maximum Likelihood Estimation

Als Nächstes stellt sich die Frage, wie die Gewichte des neuronalen Netzes gefunden werden können. Eine Methode dafür ist die Maximum-Likelihood Methode. Für unser Problem der Klassifikation von Hunde- und Katzenbildern können wir ein neuronales Netzwerk erstellen, das einer Wahrscheinlichkeitsverteilung über die Klasse entspricht. Das kann dadurch erreicht werden, indem nur ein Perzeptron im letzten Layer verwendet wird, dass die Sigmoid Funktion verwendet. Die Ausgabe des neuronalen Netzes wird dann eine Zahl im Intervall $(0, 1)$ sein und interpretiert die Zahl als die Wahrscheinlichkeit, dass das Netzwerk die Klasse „Hund“ vorhersagt, $p(Klasse = Hund|Bild, Gewichte)$. Die Wahrscheinlichkeit, dass das Netzwerk „Katze“ für dieses Bild vorhersagt, ist dann die Gegenwahrscheinlichkeit $p(Klasse = Katze|Bild, Gewichte) = 1 - p(Klasse = Hund|Bild, Gewichte)$. Hier wird auf die Gewichte bedingt, da die Wahrscheinlichkeit, die das Netzwerk der Klasse gibt, immer abhängig von den Gewichten des Netzwerks ist. (Goodfellow, 2016, S. 131)

Die Grundidee hinter der Maximum-Likelihood Methode ist, dass mit maximaler Wahrscheinlichkeit die Bilder im Datensatz richtig klassifiziert werden. Da für unseren Datensatz angenommen werden kann, dass die einzelnen Bilder unabhängig voneinander sind, weil sie insbesondere keine Videos oder Duplikate enthalten, ist diese Wahrscheinlichkeit das Produkt der einzelnen Wahrscheinlichkeiten für jedes einzelne Bild. Das bedeutet, dass folgendes Optimierungsproblem gelöst werden muss:

$$\max_{\text{Gewichte}} \prod_{i=1}^N p(\text{Klasse} = \text{richtige Klasse}_i | \text{Bild}_i, \text{Gewichte}),$$

wobei *richtige Klasse_i* und *Bild_i* die Klasse und das Bild des *i*-ten Elements des Datensatzes mit insgesamt *N* Bildern ist und Π das Produkt über jedes Element darstellt. Für die Optimierung ist es oft hilfreich, das Problem als Minimierung über eine Summe darzustellen. Das kann erreicht werden, indem der natürliche Logarithmus angewandt und die negative Kostenfunktion minimiert wird:

$$\min_{\text{Gewichte}} \sum_{i=1}^N -\ln(p(\text{Klasse} = \text{richtige Klasse}_i | \text{Bild}_i, \text{Gewichte}))$$

Um diese Formel weiter zu vereinfachen, kann man für jedes Element des Datensatzes die Zahl y_i definieren, die null ist, wenn die zugrunde liegende Klasse „Katze“ ist und eins ist, wenn die Klasse „Hund“ ist. Damit kann die Wahrscheinlichkeit für die richtige Klasse folgendermaßen geschrieben werden:

$$\begin{aligned} p(\text{Klasse} = \text{richtige Klasse}_i | \text{Bild}_i, \text{Gewichte}) &= \\ p(\text{Klasse} = \text{Hund} | \text{Bild}_i, \text{Gewichte})^{y_i} \cdot p(\text{Klasse} = \text{Katze} | \text{Bild}_i, \text{Gewichte})^{1-y_i} &= \\ p(\text{Klasse} = \text{Hund} | \text{Bild}_i, \text{Gewichte})^{y_i} \cdot (1 - p(\text{Klasse} = \text{Hund} | \text{Bild}_i, \text{Gewichte}))^{1-y_i} &= \\ f(\text{Bild}_i)^{y_i} \cdot (1 - f(\text{Bild}_i))^{1-y_i}. \end{aligned}$$

Hier ist $f(\text{Bild}_i)$ die Ausgabe des neuronalen Netzes, also die Wahrscheinlichkeit, dass das Netz „Hund“ vorhersagt. Eingesetzt in das Minimierungsproblem und unter Nutzung der Logarithmus Regeln, wird letztendlich folgendes Optimierungsproblem gelöst:

$$\min_{\text{Gewichte}} \sum_{i=1}^N -(y_i \ln(f(\text{Bild}_i)) + (1 - y_i) \ln(1 - f(\text{Bild}_i))) = \min_{\text{Gewichte}} \sum_{i=1}^N E_i.$$

Diese Kostenfunktion wird auch Binary-Cross-Entropy (Binäre Kreuzentropie) genannt und ist die Funktion, die in dieser Arbeit für die Klassifikation von Hunden und Katzen verwendet wird. (Goodfellow, 2016, S. 131-133)

2.5 Optimizer

Während die Theorie des Optimierungsproblems im letzten Kapitel beschrieben wurde, wird in diesem Kapitel erklärt, wie die Gewichte in der Praxis mit passenden Optimierungsalgorithmen gefunden werden können. Die Lösung dafür sind Optimizer. Diese minimieren durch Anpassung der Gewichte die Verlustfunktion. Die meisten der Optimierungsalgorithmen befassen sich mit der Minimierung. (Aston Zhang, 2021, S. 428)

2.5.1 Gradientenabstiegsverfahren

Die am häufigsten verwendeten Algorithmen zur Optimierung von neuronalen Netzen sind Versionen des Gradientenabstiegsverfahrens. In jedem Schritt wird versucht die Funktion zu minimieren, indem der aktuelle Punkt in Richtung des steilsten Abstiegs verschoben wird:

$$w^{(i+1)} = w^i - \alpha \nabla E_{\alpha}$$

hier ist alpha die Länge eines Schrittes. Im Bereich des Maschinellen Lernens wird sie auch als Lernrate bezeichnet. w^i ist der Vektor in dem alle Gewichte des neuronalen Netzes enthalten sind. (Bishop, 2009, S. 240)

Ziel des Gradientenabstiegsverfahrens ist es in Richtung der lokalen Optima zu konvergieren. Entscheidend dabei ist die richtige Wahl der Lernrate. Dies erweist sich jedoch als sehr kompliziert. Somit wird die optimale Lernrate häufig durch „trial and error“ bestimmt. Wird sie zu klein gewählt kann keine deutliche Verbesserung in jedem Schritt erzielt werden und sie konvergiert sehr langsam. Wählt man sie hingegen zu groß kann es passieren, dass das Minimum übersprungen wird und die Methode überhaupt nicht konvergieren kann. (Murphy, 2012, S. 247)

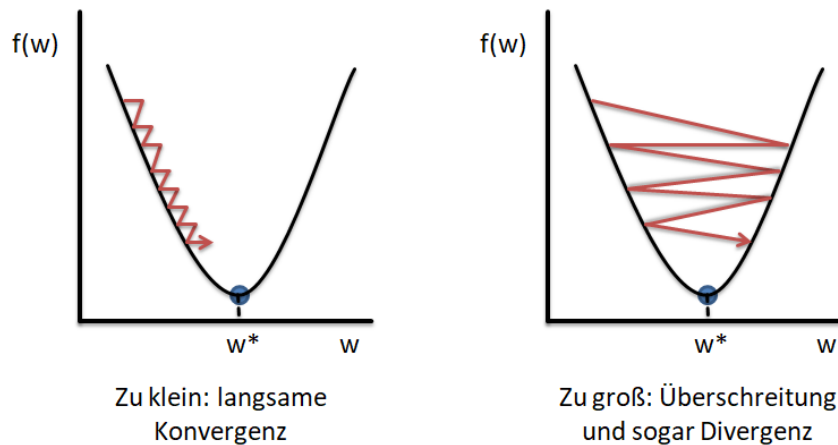


Abb. 4: Gradientenabstieg und passende Lernrate

Für die Optimierungsverfahren, die auf dem Gradientenabstiegsverfahren aufbauen wird der Gradient benötigt. Eine Methode, um den Gradienten für neuronale Netze zu berechnen ist Backpropagation. Wenn das Netzwerk auf einer Eingabe angewandt wird, werden die Zwischenergebnisse der einzelnen Schichten gespeichert. Danach werden sie verwendet, um die Gradienten zu berechnen. Währenddessen wird das Netzwerk in umgekehrter Reihenfolge, von der Ausgangs- zur Eingabeschicht durchlaufen, da es sich beim Netzwerk um eine Verkettung mehrerer Funktionen handelt, deren Ableitung bekannt ist. Um die Gradienten zu berechnen, kann die Kettenregel angewendet werden. (Goodfellow, 2016, S. 204). Moderne Deep Learning Bibliotheken wie PyTorch, Tensorflow und Jax können den Gradienten von den meisten Funktionen auf diese Art und Weise selbstständig berechnen. (PyTorch, 2022; Tensorflow, 2022; Jax, 2022)

2.5.2 Stochastisches Gradientenabstiegsverfahren

Ein Problem beim Training neuronaler Netze ist die Menge der Daten, die während eines Schrittes des Gradientenabstiegsverfahrens verwendet wird. Bei einer großen Menge an Bildern und umfangreichen Netzwerken ist diese Datenmenge für moderne Computer zu groß. Die gängigste Methode ist daher der stochastische Gradientenabstieg. Bei jedem Schritt des Algorithmus wird eine „Ministichprobe“ von Beispielen genommen. Diese Beispiele sind zufällig ausgewählt und werden Batch genannt. Die Gewichte werden dann mit jedem einzelnen Batch angepasst:

$$w^{(i+1)} = w^{(i)} - \alpha \nabla \sum_{i \in B} E_i,$$

wobei B die Indizes der Batch enthalten. Im Folgenden wird der Gradient für die aktuelle Batch als $g = \nabla \sum_{i \in B} E_i$. Bezeichnet. Die Laufzeit des Gradientenabstiegs ist folglich unabhängig von der Anzahl der Trainingsdaten. (Goodfellow, 2016, S. 151-152)

2.5.3 Momentum

Ein Problem bei dem Stochastischen Gradientenabstieg ist jedoch, dass die einzelnen Schritte nicht immer in die Richtung des steilsten Abstiegs gehen. Dadurch ist die Konvergenz oft langsam. Um das zu lösen, berücksichtigt Momentum die vergangenen Gradienten, um die Aktualisierung zu verfeinern. Dies zeigt sich in der variablen V , die ein exponentiell gewichteter Durchschnitt der Gradienten der vorliegenden Schritte ist. (ML Glossary, 2017)

$$v^{(i+1)} = \beta v^{(i)} + (1 - \beta)g$$

$$w^{(i+1)} = w^{(i)} - \alpha v^{(i+1)}$$

$v^{(i+1)}$ ist der exponentiell gewichtete Durchschnitt der Gradienten. β ist ein Hyperparameter und α ist die Lernrate. (Ilya Sutskever, 2013)

2.5.4 RMSProp

RMSProp ist ein weiterer adaptiver Algorithmus zur Optimierung der Lernrate und Stabilisierung des Trainings, welcher mit einem exponentiell- gewichteten Durchschnitt der Quadrate der vergangenen Gradienten arbeitet. die Lernrate wird anschließend durch diesen Durchschnitt geteilt um die Konvergenz zu beschleunigen. (ML Glossary, 2017)

$$s^{(i+1)} = \beta s^{(i)} + (1 - \beta)g^2$$

$$w^{(i+1)} = w^{(i)} - \alpha \frac{g}{\sqrt{s^{(i+1)} + \varepsilon}}$$

$s^{(i+1)}$ ist der exponentiell gewichtete Durchschnitt der Gradienten. Wie bei Momentum, ist beta ein Hyperparameter und alpha die Lernrate. ε ist ein sehr kleiner Wert, um eine Division durch Null zu vermeiden. (Hinton, o. J)

2.5.5 Adam

Der Adam-Optimizer ist die Kombination der Ideen von Momentum und RMSProp. Er berechnet die adaptive Lernrate für jeden Parameter. Zunächst wird der exponentiell gewichtete Durchschnitt der vergangenen Gradienten berechnet. Daraufhin berechnet er den exponentiell gewichteten Durchschnitt der Quadrate der vergangenen Gradienten. Diese Mittelwerte haben eine Verzerrung gegen Null, und um dem entgegenzuwirken, wird eine Verzerrungskorrektur vorgenommen. Abschließend werden die Parameter anhand der Informationen aus dem berechneten Durchschnittswert aktualisiert. (ML Glossary, 2017)

$$v^{(i+1)} = \beta_1 v^{(i)} + (1 - \beta_1)g$$

$$s^{(i+1)} = \beta_2 s^{(i)} + (1 - \beta_2)g^2$$

$$v_{corrected}^{(i+1)} = \frac{v^{(i+1)}}{1 - \beta_1^{i+1}}$$

$$s_{corrected}^{(i+1)} = \frac{s^{(i+1)}}{1 - \beta_2^{i+1}}$$

$$W = W - \alpha \frac{v_{corrected}^{(i+1)}}{\sqrt{s_{corrected}^{(i+1)} + \epsilon}}$$

$v^{(i+1)}$ ist der exponentiell gewichtete Durchschnitt der vergangenen Gradienten, $s^{(i+1)}$ der exponentiell gewichtete Durchschnitt der vergangenen Quadrate der Gradienten. β_1 und β_2 stehen für die abzustimmenden Hyperparameter und ϵ ist ein sehr kleiner Wert, um eine Division durch Null zu vermeiden. α ist die Lernrate. (Diederik P. Kingma, 2015)

2.6 Overfitting

Eine große Herausforderung beim maschinellen Lernen ist das Entwickeln eines Algorithmus, der nicht nur bei den Trainingsdaten, sondern auch bei neuen Eingaben gut abschneidet. Das Modell fängt an den Trainingsdatensatz auswendig zu lernen und funktioniert somit nicht mehr gut auf den Test Daten. Dieses Problem nennt man Overfitting. Overfitting lässt sich an

einer großen Lücke zwischen Trainings und Testfehlern erkennen. (Goodfellow, 2016, S. 259)

Der Trainingsdatensatz wird in Test-Set und Validierung-Set unterteilt, wobei die Größe des Validierungsdatensatzes 25% der Größe vom gesamten Datensatz entspricht. Grund dafür ist eine Möglichkeit das Problem des Overfittings zu verhindern. Early Stopping funktioniert durch einen frühzeitigen Abbruch des Lernprozesses. Es werden Trainingsfehler und Validierungsfehler miteinander verglichen. Während der Trainingsfehler der Optimierung verkleinert wird, nimmt der Validierungsfehler ab einem Punkt zu. Early Stopping ist eine heuristische Methode, die das Training abbricht, wenn sich der Validierungsfehler für einige Epochen nicht mehr verbessert. Ab dem Punkt, an dem das Modell sich auf dem Validierungsdatensatz nicht mehr verbessert, kann der Trainingsprozess gestoppt werden. Durch diesen frühzeitigen Stopp wird die Überanpassung des Netzwerks reduziert. (Bishop, 2009, S. 259)

3 Convolutional Neuronal Network

Um die Probleme in der Bildverarbeitung zu lösen, werden heutzutage Convolutional Neuronal Networks verwendet. Das Netzwerk ist invariant gegenüber Verschiebungen, somit eignet es sich gut für die Bilderkennung. Der Name Convolutional Neuronal Network weist darauf hin, dass das Netzwerk eine mathematische Operation namens Faltung (engl. convolution) verwendet. Normalerweise entspricht die in einem neuronalen Faltungsnetz verwendete Operation nicht genau der mathematischen Definition, sondern eher einer Kreuzkorrelation. Der Unterschied wird in Appendix B erklärt. (Goodfellow, 2016, S. 330)

3.1 Eingabe

Tensoren sind in der linearen Algebra eine Art von Datenstruktur. Ein Tensor ist eine Verallgemeinerung von Vektoren und Matrizen und kann als mehrdimensionales Array gesehen werden. Im Vergleich zu Vektoren und Matrizen haben Tensoren aber nicht eine oder zwei, sondern beliebig viele Axen. Ein Vektor kann als Tensor mit einer Dimension und eine Matrix kann als zwei-dimensionaler Tensor beschrieben werden. Sie sind eine Möglichkeit die Eingangsdaten zu speichern. Gewichte und verschiedene Bias aus denen das neuronale Netz besteht, werden auch in Tensoren gespeichert.

Die Bilder, die in der Bilderkennung verwendet werden sind häufig farbig, was dafür sorgt, dass es mehrere Kanäle gibt. Farbige Bilder werden normalerweise in drei Kanäle unterteilt: Rot, Grün und Blau. Insgesamt ist jedes Bild also ein dreidimensionaler Tensor mit der Form Anzahl der Kanäle x Höhe x Breite. Jede Schicht eines Convolutional Neuronal Networks hat als Eingabe und Ausgabe einen dreidimensionalen Tensor.

3.2 Faltungsschicht

Die Faltungsschicht wird im Folgenden anhand eines Beispiels erklärt.

Abb. 5 ist die Eingabe, ein zwei dimensionaler Tensor mit einer Höhe von 3 und einer Breite von 3. Die Form des Tensors wird mit 3×3 oder $(3,3)$ gekennzeichnet. Die Höhe und Breite des Gewichts sind beide 2 also 2×2 . Bei der Faltungsschicht wird das Gewicht zunächst an der oberen linken Ecke des Eingabetensors positioniert. Zuerst wird es von links nach rechts geschoben, dann ein Element nach unten und wieder von links nach rechts geschoben. Das wird wiederholt, bis der Gewichtstensor rechts unten angekommen ist. Wenn der Gewichtstensor an eine bestimmte Position gelangt, werden der in diesem Tensor enthaltenen Eingabe-Subtensor und Kernel Tensor elementweise miteinander multipliziert. Der resultierende Tensor wird summiert, was den skalaren Wert an diesem Feld ergibt. Wenn mehrere Kanäle vorhanden sind, dann wird die Operation kanalweise ausgeführt und daraufhin summiert. Um mehrere Kanäle im Output zu generieren, kann diese Operation mit mehreren Gewichtstensoren wiederholt werden. (Aston Zhang, 2021, S. 234-235)

Input		Kernel		Output																	
<table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td>0</td><td>1</td><td>2</td></tr> <tr><td>3</td><td>4</td><td>5</td></tr> <tr><td>6</td><td>7</td><td>8</td></tr> </table>	0	1	2	3	4	5	6	7	8	*	<table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td>0</td><td>1</td></tr> <tr><td>2</td><td>3</td></tr> </table>	0	1	2	3	=	<table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td>19</td><td>25</td></tr> <tr><td>37</td><td>43</td></tr> </table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

Abb. 5: Kreuzkorrelation Beispiel

In Abb. 5 gilt dann:

$$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19,$$

$$1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 = 25,$$

$$3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 = 37,$$

$$4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 = 43.$$

Hier hat der Ausgabtensor eine Höhe und Breite von 2 und die vier Elemente ergeben sich aus der zweidimensionalen Kreuzkorrelation.

3.2.2 Padding

Ein Problem bei der Anwendung von Faltungsschichten besteht darin, dass Pixel am Rande unseres Bildes verloren gehen. Die Pixel in den Ecken werden kaum genutzt. Da normalerweise kleine Kernel verwendet werden, gehen bei einer beliebigen Faltung möglicherweise nur ein paar Pixel verloren. Diese Zahl summiert sich jedoch, wenn wir viele aufeinanderfolgende Faltungsschichten anwenden. Eine simple Lösung für dieses Problem ist das Hinzufügen zusätzlicher Füllpixel am Rand des Eingabebildes, wodurch die effektive Größe des Bildes erhöht wird. In der Regel wird der Wert für alle zusätzlichen Pixel auf null gesetzt. (Aston Zhang, 2021, S. 241)

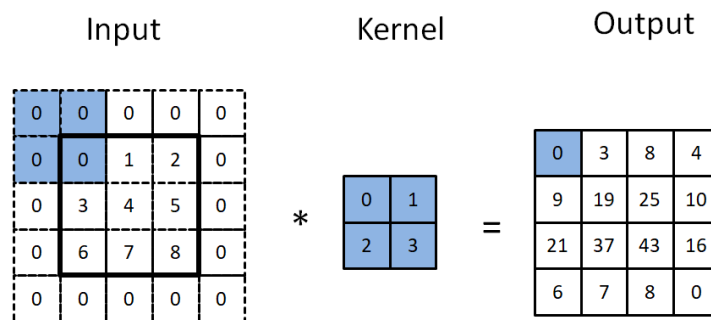


Abb. 6: Padding und Kernel Beispiel

In Abb 6. Wird eine 3x3 Eingabe aufgefüllt, indem ihre Größe auf 5x5 erhöht wird. Die entsprechende Ausgabe wird dann zu einer 4x4 Matrix. Die schattierten Bereiche sind das erste Ausgabeelement sowie die Eingabe- und Kernel-Tensorelemente, die für die Berechnung der Ausgabe verwendet werden: $0 \times 0 + 0 \times 1 + 0 \times 2 + 0 \times 3 = 0$.

3.2.3 Stride

Abgesehen vom Padding kann auch der Stride von einer Faltungsschicht verändert werden um die Höhe und Breite des Ausgabetensors zu verkleinern. Stride ist die Schrittgröße, also wie viele Pixel nach links oder nach unten gesprungen werden. Beispielsweise bedeutet, ein Stride von 2 bedeutet, dass nur jeder zweite Pixel Verwendet wird und der Tensor halb so hoch und breit ist. (Goodfellow, 2016, S. 705)

3.3 Pooling

Eine häufig verwendete Methode, um irrelevante Informationen zu beseitigen und sich mehr auf die wichtigsten Details zu konzentrieren, ist das Pooling. Wie die Faltungsschichten bestehen die Pooling-Operatoren aus einem Fenster mit fester Größe, das entsprechend über alle Eingaben geschoben wird. Im Gegensatz zu Kreuzkorrelationsberechnung der Eingaben mit einem Kernel enthält die Pooling-Schicht keine Gewichte die gelernt werden. Pooling Operatoren berechnen entweder den Maximal- oder den Durchschnittswert der Elemente im Pooling Fenster. Diese Vorgänge werden als Maximum-Pooling bzw. Average-Pooling bezeichnet. (Aston Zhang, 2021, S. 249-250)

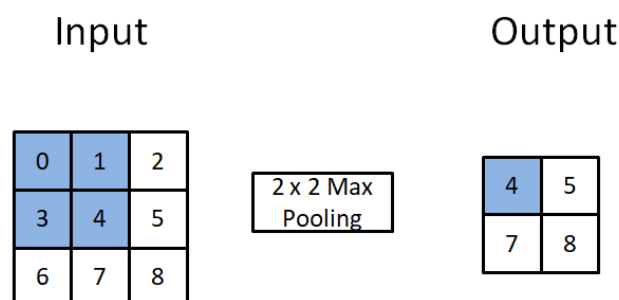


Abb. 7: Maximum Pooling Beispiel

$$\max(0, 1, 3, 4) = 4, \max(1, 2, 4, 5) = 5, \max(3, 4, 6, 7) = 7, \max(4, 5, 7, 8) = 8.$$

4 Praktisches Beispiel

4.1 Aufgabe

Nachdem bisher alle notwendigen Grundlagen erarbeitet wurden, soll im Rahmen dieser Arbeit ein eigenes künstliches neuronales Netz trainiert werden. Hierbei handelt es sich um ein Convolutional Neural Network, welches in der Lage sein soll, Katzenbilder von Hundebildern eigenständig zu unterscheiden und richtig zu identifizieren. Ziel dabei ist es eine möglichst hohe Genauigkeit im Unterscheiden der Bilder zu erreichen.

4.2 Daten

Wie bereits am Anfang dieser Arbeit erwähnt gilt, je mehr Trainingsdaten, desto bessere Ergebnisse erzielt das neuronale Netz im Allgemeinen. (Tom B. Brown, 2020). Aus diesem Grund wurde ein umfangreicher Open-source Trainings Datensatz der Website Kaggle verwendet. (Dogs vs. Cats Redux: Kernels Edition, 2016). Der Datensatz enthält 25.000 Bilder von Hunden und Katzen. Die Bilder liegen alle im JPEG Format vor. Davon sind die ersten 12.500 Bilder von Katzen und die restlichen 12.500 Bilder von Hunden. Jedes Bild in diesem Ordner hat die Bezeichnung als Teil des Dateinamens. Ein weiterer Grund für die Auswahl dieses Datensatzes ist, dass er viele Verschiedene Arten von Bildern enthält z.B. mit Störenden Objekten, mit Menschen im Hintergrund, überbelichtete Bilder usw. Die Aufzählung der Bilder ist Numerisch und beginnt mit Null. Bevor die Bilder im Convolutional Network verwendet werden können müssen sie in einer einheitliche Größe vorliegen. Dafür werden die Bilder auf ein 256x256 Format reduziert. Außerdem sind einige Bilder in dem Datensatz im Querformat und die anderen wiederum im Hochformat, deswegen wird auf das Zentrum des Bildes zugeschnitten (auf 224x224) um das Hauptobjekt im Fokus zu haben. Es stellt sich die Frage: woher das Programm weiß bei welchem Bild es sich nun um eine Katze oder um einen Hund handelt. Wie bereits oben erwähnt enthält jedes Bild den dazugehörigen Titel im Namen der Datei. So hat jedes Bild mit einer Katze ein „cat“ im Namen und jedes Bild mit einem Hund ein „dog“ im Namen. Der gesamte Datensatz hat eine Größe von 1,82 Gigabyte.

4.3 Verwendete Programmiersprache & Bibliotheken

Als Programmiersprache wird Python verwendet, diese ist eine Objektorientierte Programmiersprache mit einer einfachen Syntax. Außerdem enthält Python eine Vielzahl an Bibliotheken, die für das Maschinelle Lernen sehr hilfreich sind. (Python, 2022). Eine dieser Bibliotheken für Python ist PyTorch. PyTorch wurde von der Abteilung für künstliche Intelligenz von Meta entwickelt. Diese Open-Source-Bibliothek ist sehr beliebt für die Erstellung von künstlichen Intelligenzen. Sie basiert auf Torch und wurde entwickelt um mehr Flexibilität und höhere Geschwindigkeiten für die Implementierung neuronaler Netze zu bieten. (PyTorch, 2022; Torch, 2022). Um die Bilder effizient zu verwenden, wird Torchvision benutzt. Diese Bibliothek ist für Computer Vision zuständig und beinhaltet gängige Bildtransformation. Sie gehört zu PyTorch. (PyTorch, 2022). Damit effizientes Rechnen mit großen Matrizen und Arrays möglich ist, wird die NumPy Bibliothek verwendet. NumPy ist eine Abkürzung für „Numeric Python“ und ist auf linearer Algebra aufgebaut. Sie beinhaltet Funktionen für numerische Berechnungen und erweitert mathematische Routinen. (Numpy, 2018)

4.4 Training

Das neuronale Netz wird mithilfe von Google Collab auf der Grafikkarte trainiert, um die Performance zu verbessern. Die Bilder werden zunächst in PyTorch Tensoren umgewandelt. Ein Datensatz wird erstellt, welcher die Trainingsdaten von Validierungsdaten trennt. Der Datensatz bestehend aus 25.000 Bildern, wird unterteilt in 18.750 Trainingsbilder und 6.250 Validierungsbilder. Verwendet wird die AlexNet Architektur, jedoch mit einem anstatt von 1000 Ausgaben. (Alex Krizhevsky, 2012). Das neuronale Netz mit knapp 60 Millionen Parametern besteht aus fünf Faltungsschichten. Alle dieser Schichten sind von Max-Pooling-Schichten gefolgt. Verwendet wird die ReLu Aktivierungsfunktion. Da nur zwei Klassen vorhanden sind wird der Binary- Cross Entropie Loss Genutzt. Als Optimizer wird hier der Adam-Optimizer verwendet, der ein Momentum erster und zweiter Ordnung aus den letzten Batches verwendet, um die Optimierung zu beschleunigen. (Diederik P. Kingma, 2015). Das Modell wurde auf den Trainingsdaten trainiert und nach jedem vollständigen Durchlauf durch die Trainingsdaten (Epoche) auf den Validierungsdaten getestet. Um Overfitting zu vermeiden wird das Early Stopping verwendet und das Training wird gestoppt, wenn sich die Kostenfunktion auf dem Validierungsdatensatz für zehn Epochen nicht verbessert. Die Testdaten wurden für das Training nicht verwendet.

4.5 Ergebnis

Der unabhängige Testdatensatz wurde in Kooperation mit der Computer Vision & Artificial Intelligence Gruppe der Technischen Universität München mit den ungelabelten Testdaten von Kaggle erstellt. Dazu wurde zuerst ein vortrainiertes Netzwerk auf den Daten angewandt und dann händisch durch manuelle Korrekturen von mir verbessert.

Auf diesem Datensatz konnte mein Modell eine Genauigkeit von 87,37 % erzielen. Nach 19 Epochen wurde der Trainingsprozess aufgrund des Early Stoppings abgeschlossen, da es ab diesem Zeitpunkt schon zu Overfitting kam. Abb. 8 stellt die Graphen vom Trainings und Validation Loss des neuronalen Netzes dar. Wie man erkennen kann, sinken Trainings loss und Validation loss Anfangs bis zu achten Epoche gleichmäßig ab. Ab dieser fängt der Validation loss wieder an zu steigen, wohingegen der Trainings loss weiterhin sinkt.

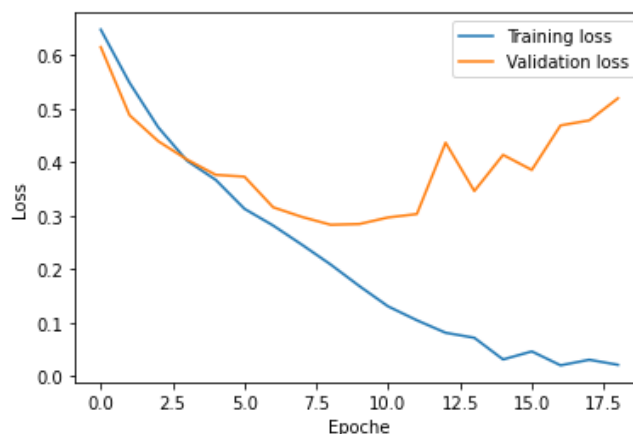


Abb. 8: Trainings & Validation loss

5 Resümee

Nachdem auf den historischen Hintergrund neuronaler Netze eingegangen wurde anschließend aufgezeigt, wie diese sich bis zum heutigen Zeitpunkt entwickelt haben. Ausschlaggebend für das Ergebnis eines neuronalen Netzes sind seine Gewichte. Zur Schätzung der Wahrscheinlichkeit wurde die Maximum Likelihood Estimation vorgestellt. Um diese Vorhersagen zu verbessern wurden Optimizer vorgestellt, welche essentiell für das Maschinelle Lernen sind. Anschließend wurde ein weit verbreitetes Problem neuronaler Netze vorgestellt, um die-

se Problem zu lösen wurde das Early Stopping aufgezeigt. Im nächsten Kapitel wurde näher auf die Erkennung und Verarbeitung von Bildern mittels Convolutional Neuronal Networks eingegangen. Nachdem alle Grundlagen für das praktische Modell erarbeitet wurden konnte Die Struktur des neuronalen Netzes zur Unterscheidung von Katzen und Hunden vorgestellt werden. Dieses Modell lag auf einem Open- Source Datensatz zu 87,3% richtig. Um die Genauigkeit noch weiter zu verbessern, könnte als nächster Schritt eine größere und verbesserte Netzwerkarchitekturen wie Residual Networks oder Vision Transformer verwendet werden und der Datensatz durch Data Augmentation künstlich vergrößert werden. (Sun, 2015; Ekin D. Cubuk, 2019)

Literaturverzeichnis

Alex Krizhevsky, I. S. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*.

Andrew Brock, S. D. (2021). *High-Performance Large-Scale Image Recognition Without Normalization*.

Aston Zhang, Z. C. (2021). *Dive into Deep Learning*.

Bishop, C. (2009). *Pattern recognition and machine learning. Information science and statistics*.

Diederik P. Kingma, J. L. (2015). *Adam: A Method for Stochastic Optimization*.

Dogs vs. Cats Redux: Kernels Edition. (2016). Abgerufen am 12. 5 2022 von Kaggle:
<https://www.kaggle.com/competitions/dogs-vs-cats-redux-kernels-edition>

Ekin D. Cubuk, B. Z. (2019). *RandAugment: Practical automated data augmentation*.

Goodfellow, I. B. (2016). *Deep Learning*. MIT Press.

Gregor Kemper, F. R. (2022). *Lineare Algebra Mit einer Einführung in diskrete Mathematik und Mengenlehre*. Springer.

Hinton, G. (o. J). *Neural Networks for Machine Learning, Overview of mini-batch gradient descent*.
Abgerufen am 12. 10 2022 von
https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf

- Ilya Sutskever, J. M. (2013). *On the importance of initialization and momentum in deep learning*.
- Jax. (2022). Abgerufen am 12. 10 2022 von <https://github.com/google/jax>
- Kaiming He, X. Z. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*.
- Kriesel, D. (2005). *Ein kleiner Überblick über Neuronale Netze*.
- ML Glossary. (2017). Abgerufen am 12. 10 2022 von <https://ml-cheatsheet.readthedocs.io/en/latest/optimizers.html>
- Murphy, K. P. (2012). *Machine Learning A Probabilistic Perspective*. London: MIT Press.
- Niall O' Mahony, S. C. (2019). *Deep Learning vs. Traditional Computer Vision*. Institute of Technology Tralee.
- Numpy. (2018). Abgerufen am 3. 11 2022 von Bigdata-insider: <https://www.bigdata-insider.de/was-ist-numpy-a-735687>
- Python. (2022). Abgerufen am 20. 08 2022 von <https://www.python.org/>.
- PyTorch. (2022). Abgerufen am 20. 08 2022 von <https://pytorch.org/>
- Rosenblatt, F. (1958). The perceptron a probabilistic model for information storage an organization in the brain. *Psychological Review* 65 Nr. 6.
- Rosenblatt, F. (1958). *The Perceptron: a probabilistic model for information storage and organization in the brain*.
- Sun, K. H. (2015). *Deep Residual Learning for Image Recognition*.
- Tensorflow. (2022). Abgerufen am 20. 10 2022 von <https://www.tensorflow.org/>
- Tom B. Brown, B. M.-V. (2020). *Language Models are Few-Shot Learners*.
- Torch. (2022). Abgerufen am 21. 10 2022 von <https://github.com/torch/torch7>

Appendix A

A.1 Mathematischer Hintergrund

Zunächst werden die mathematischen Hintergründe erklärt, die für das Verständnis von neuronalen Netzen benötigt werden. Diese umfassen Matrizen inklusive der Matrixmultiplikation, die für die einfache Schreibweise von Mehrschichtigen Perzeptronen verwendet werden, und Gradienten, die wichtig für das Gradientenabstiegsverfahren sind.

A.1.1 Matrizen

In der linearen Algebra ist eine Matrix eine rechteckige Anordnung von Zahlen oder Ausdrücken in Zeilen und Spalten. Eine Matrix mit m Zeilen und n Spalten hat die Dimension $m \times n$. Eine 3×4 -Matrix kann wie folgt geschrieben werden

$$M = \begin{bmatrix} M_{11} & M_{12} & M_{13} & M_{14} \\ M_{21} & M_{22} & M_{23} & M_{24} \\ M_{31} & M_{32} & M_{33} & M_{34} \end{bmatrix}$$

diese 3×4 -Matrix veranschaulicht die Standardschreibweise von Matrizen. Die Zahlen werden in einem Gitter angeordnet und von Klammern umgeben. Eine Matrix die nur eine Zeile ($m = 1$) oder eine Spalte ($n = 1$) hat, wird als Vektor definiert. Ein Zeilen- bzw. Spaltenvektor der Dimension n kann als $n \times 1$ Matrix bezeichnet werden. Eine Matrix kann auch mit einer weiteren Matrix bzw. Vektor multipliziert werden. Eine $m \times n$ Matrix A kann mit einer $n \times r$ Matrix B multipliziert werden. Das Ergebnis, AB , ist eine $m \times r$ Matrix. $M \times r$, die folgende Elemente enthält:

$$(AB)_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

Transponiert bedeutet, dass eine $m \times n$ Matrix zu einer $n \times m$ Matrix wird und das ij -te Element an der ji -ten Stelle steht. Nehmen wir an, dass A eine 3×2 -Matrix ist und B eine 2×7 -Matrix ist. Dann ist AB eine 3×7 -Matrix. Das Element an Position ij ist die Summe der Produkte aus den Elementen in der i -ten Zeile der ersten Matrix A und den entsprechenden Elementen in der j -ten Spalte der zweiten Matrix B . Zu beachten ist, dass das Produkt AB nur

dann definiert ist, wenn die obige Bedingung erfüllt ist, d. h. die Anzahl der Spalten der ersten Matrix muss der Anzahl der Zeilen der zweiten Matrix entsprechen. (Gregor Kemper, 2022, S. 101-102, 131)

A.1.2 Gradient

Ein Gradient ist die Ableitung einer Funktion $f: \mathbb{R}^n \rightarrow \mathbb{R}$, die einen Vektor auf einen Skalar abbildet. Der Gradient ist ein Vektor der Größe n , der die partiellen Ableitungen enthält. Eine i -te partielle Ableitung ist dabei die Ableitung der Funktion nach dem i -ten Element der Eingabe der Funktion, während alle anderen Eingabe-Werte als Konstanten betrachtet werden. (Kriesel, 2005, S. 79)

Appendix B

B.1 Faltung

Um die Faltung zu verdeutlichen ein kleines Beispiel:

Nehmen wir an, wir verfolgen die Position eines Raumschiffs mit einem Lasersensor. Unser Lasersensor liefert eine einzige Ausgabe $x(t)$, Die Position des Raumschiffs zum Zeitpunkt t . Sowohl x als auch t sind reelwertig d.h wir können zu jedem Zeitpunkt einen anderen Messwert vom Laser erhalten. Nehmen wir nun an das unser Lasersensor etwas gestört ist. Um eine weniger gestörte Position des Raumschiffs zu erhalten möchten wir den Mittelwert aus mehreren Messungen berechnen. Natürlich sind die neuen Messungen relevanter, also wollen wir einen gewichteten Mittelwert bilden, der den jüngsten Messungen mehr Gewicht verleiht. Dies können wir mit einer Gewichtsfunktion $w(a)$ erreichen, wobei a das Alter einer Messung ist. Wenn wir zu jedem Zeitpunkt eine solche Durchschnittsfunktion anwenden erhalten wir eine neue Funktion, die s eine geglättete Schätzung der Position des Raumschiffs liefert.

$$s(t) = \int x(a)w(t-a) da$$

Dieser Vorgang nennt sich Faltung. In dem Beispiel sind einige Einschränkungen vorhanden. Die Faltung ist im Allgemeinen für alle Funktionen definiert, für die das obige Integral defi-

niert ist und kann für andere Zwecke als Bildung gewichteter Mittelwerte verwendet werden. Dieses Beispiel wurde leicht abgeändert von (Goodfellow, 2016, S. 331) übernommen.

B.1.2 Kreuzkorrelation

Normalerweise entspricht die in einem neuronalen Faltungsnetz verwendete Operation nicht genau der mathematischen Definition, sondern der Kreuzkorrelation.

$$s(t) = \int x(a)w(t + a) da$$

Die Formel für die Kreuzkorrelation ist sehr ähnlich zu der Formel der Faltung. Der einzige Unterschied ist, dass die Gewichtung bei der Faltung gespiegelt wird und bei der Kreuzkorrelation nicht. (Aston Zhang, 2021, S. 234-235)

Anhang

Das neuronale Netz zur Unterscheidung von Katzen und Hunden ist für jeden öffentlich zugänglich und unter folgendem Link heruntergeladen werden:

<https://github.com/theNoobSkill/wseminar>

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Hilfsmittel verwendet habe. Insbesondere versichere ich, dass ich alle wörtlichen und sinngemäßen Übernahmen aus anderen Werken als solche kenntlich gemacht habe. Mir ist bekannt, dass die vorliegende Arbeit mit 0 Punkten bewertet werden kann, wenn gegen diese Grundsätze verstoßen wurde, und dass in diesem Fall eine Zulassung zum Abitur nicht möglich ist.

München, 8. Nov 2022